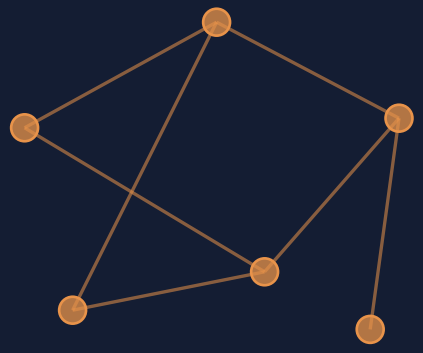
ÖZYİNELEME (RECURSION)

5. Hafta — Call Stack ile Fonksiyonların Kendini Çağırması

Karadeniz Teknik Üniversitesi

Yazılım Geliştirme Bölümü

Dr. Öğr. Üyesi Sayyad Alizadeh



Geçen Haftadan Bağlantı

- Stack ADT: push ile ekleme, pop ile çıkarma — LIFO.
- Fonksiyon çağrı yığını (call stack): her fonksiyon çağrısı bir stack frame olarak push edilir.
- **Çok derin çağrı zincirleri stack overflow'a yol açar.**
- **Bu hafta: bir fonksiyonun KENDİSİNİ çağırması — ÖZYİNELEME — bu call stack mekanizmasını birebir kullanır.**

Bu Hafta Neden Önemli?

Özyineleme, “sihirli” bir şey değildir — sadece normal fonksiyon çağrılarının stack üzerinde birikmesidir.

Bu haftanın merkezinde, her çağrının call stack'te NASIL bir frame oluşturduğunu adım adım görmek var.

Bu Haftanın Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler...

1. Özyinelemeyi ve base case/recursive case kavramlarını açıklayabilir
2. Bir özyinelemeli çağrının call stack üzerinde adım adım nasıl ilerlediğini çizebilir
3. Faktöriyel, Fibonacci ve Hanoi Kuleleri'ni özyinelemeli olarak C'de yazabilir
4. Naive Fibonacci'nin neden verimsiz olduğunu çağrı ağacıyla açıklayabilir
5. Yaygın base case hatalarını tanıyıp düzeltebilir
6. Özyineleme ile yinelemeyi doğru kriterlerle karşılaştırabilir

Bu Haftanın Yol Haritası

ÖZYİNELEME TEMELLERİ

Base case, recursive case, call stack ve stack frame kavramı

Slayt 5–28

ÖRNEKLER

Faktöriyel, Fibonacci, doğrusal toplam, Hanoi Kuleleri

Slayt 29–51

HATALAR + KARŞILAŞTIRMA + KAPANIŞ

Yaygın hatalar, recursion vs iteration, özet, quiz, ödev

Slayt 52–64

Özyineleme (Recursion) Nedir?

Bir fonksiyonun, çözümün bir parçası olarak KENDİSİNİ çağırmasıdır.

- Büyük bir problemi, AYNI TÜRDEN ama DAHA KÜÇÜK bir alt problemi çözerek çözeriz.
- Örnek: n! hesaplamak için (n-1)! hesaplayıp n ile çarparız.
- *Bu, matematikteki tümevarımla (induction) yakından ilişkilidir.*

Bir Fonksiyon Kendini Çağırdığında Ne Olur?

- Derleyici açısından özel bir şey yoktur — fonksiyon, başka bir fonksiyonu çağırıyormuş gibi kendisini çağırır.
- **Her çağrı, KENDİ parametreleri ve yerel değişkenleriyle YENİ bir stack frame alır.**
- Önceki çağrının frame'i call stack'te BEKLER — kaybolmaz.

Bu haftanın tamamı, bu tek fikri somut örneklerle görmek üzerine kurulu: her çağrı = yeni bir frame.

Base Case (Taban Durum)

- **Özyinelemenin DURDUĞU, artık kendini çağırmadığı en basit durumdur.**
- Her özyinelemeli fonksiyonun EN AZ BİR base case'i olmalıdır.

```
faktoriyel(0) = 1
```

Base case, çözümü DOĞRUDAN bilinen, başka hiçbir çağrıya ihtiyaç duymayan durumdur. Faktöriyelde bu, 0! = 1 olgusudur — matematiksel bir tanımdır, hesaplanmaz.

Recursive Case (Özyinelemeli Durum)

- **Fonksiyonun KENDİSİNİ, DAHA KÜÇÜK bir girdiyle çağırdığı durumdur.**
- Her çağrıda problem base case'e doğru KESİNLİKLE küçülmelidir.

```
faktoriyel(n) = n × faktoriyel(n - 1)
```

n-1, n'den KESİNLİKLE küçüktür, bu yüzden tekrarlanan çağrılar er geç base case'e (n=0) ulaşır. Eğer bu küçülme garanti edilmezse, sonsuz özyineleme oluşur — bunu birazdan göreceğiz.

Durma Koşulu — İkisi Bir Arada

```
int faktoriyel(int n) {
    if (n == 0) {           // BASE CASE
        return 1;
    }
    return n * faktoriyel(n - 1);   // RECURSIVE CASE
}
```

Bu iki parça birlikte çalışır: recursive case problemi KÜÇÜLTÜR, base case KÜÇÜLMEYİ DURDURUR. Biri eksikse, özyineleme ya hiç durmaz ya da yanlış çalışır.

Sonsuz Özyineleme Riski

- **Base case eksikse VEYA hiç ulaşılamıyorsa, fonksiyon KENDİNİ SONSUZA DEK çağırır.**

... ve devam eder ...

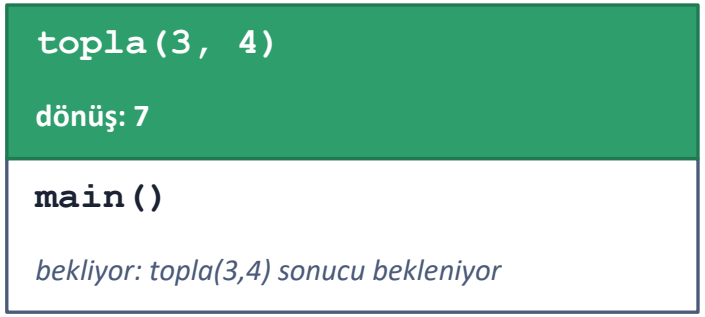


Her çağrı yeni bir stack frame ekler; bellek sınırlı olduğu için bu er geç STACK OVERFLOW ile çöker.

Call Stack Hatırlatma

4. Haftadan

- Bir fonksiyon başka bir fonksiyonu (ya da KENDİSİNİ) çağırdığında, çağrının bilgisi bir stack'e PUSH edilir.
- Çağrı tamamlanıp sonuç döndürüldüğünde, bu bilgi POP edilir ve çağrı yapan yere geri dönülür.



Özyinelemede TEK fark: çağrılan fonksiyon, çağran ile AYNI isimdir.

Stack Frame İçeriği

- **Her stack frame, en azından şu bilgileri taşır:**

Fonksiyon Adı

Hangi fonksiyonun çalıştığı (ör. faktoriyel)

Parametre Değeri

Bu çağrıya özel girdi (ör. n=3)

Bekleyen İşlem

Derin çağrı dönene kadar yapılamayan hesap (ör. 3 * faktoriyel(2))

Dönüş Değeri

Derin çağrı döndüğünde hesaplanacak sonuç

Çağrıların Derinleşmesi ve Geri Açılması (Unwinding)

Aşama	Ne Olur?	Stack'e Etkisi
Derinleşme (diving)	Her çağrı base case'e doğru ilerler	YENİ frame PUSH edilir
Base Case	Artık çağrı yapılmaz, doğrudan sonuç döner	Stack en derin noktasına ulaşır
Geri Açılma (unwinding)	Her çağrı, aldığı sonucu kullanarak KENDİ sonucunu hesaplar	Frame'ler TERS SIRAYLA POP edilir

Anahtar Fikir: Derinleşme sırasında hiçbir hesap yapılmaz — sadece iş ertelenir. Gerçek hesaplama, geri açılma sırasında, base case'ten başlayarak yukarı doğru gerçekleşir.

Faktöriyel — Matematiksel Tanım

Örnek 1

$$n! = n \times (n - 1)!$$

$$0! = 1 \qquad \text{(base case)}$$

Bu iki satır, faktoriyelin TAM ve KESİN tanımıdır — bir sonraki slaytta bunu doğrudan C koduna çevireceğiz.

Faktöriyel — Pseudocode

```
Fonksiyon Faktoriyel(n)
    eğer n == 0
        dön 1                              // base case
    dön n × Faktoriyel(n - 1)             // recursive
case
```

Faktöriyel — C Kodu

```c
int faktoriyel(int n) {
    if (n == 0) {
        return 1;                    // base case
    }
    return n * faktoriyel(n - 1);    // recursive case
}
```

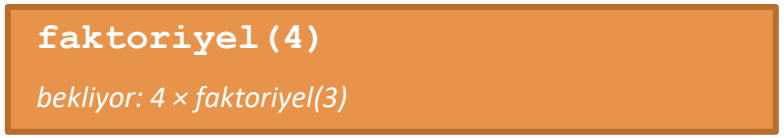
Faktöriyel — Çağrıların Matematiksel Açılımı

Bu SADECE matematiksel gösterimdir — call stack DEĞİLDİR (call stack sonraki slaytlarda)

```
faktoriyel(4)
  = 4 × faktoriyel(3)
  = 4 × (3 × faktoriyel(2))
  = 4 × (3 × (2 × faktoriyel(1)))
  = 4 × (3 × (2 × (1 × faktoriyel(0))))
  = 4 × (3 × (2 × (1 × 1)))      ← base case: faktoriyel(0)=1
  = 24
```

Call Stack Büyüyor — Adım 1 / 5

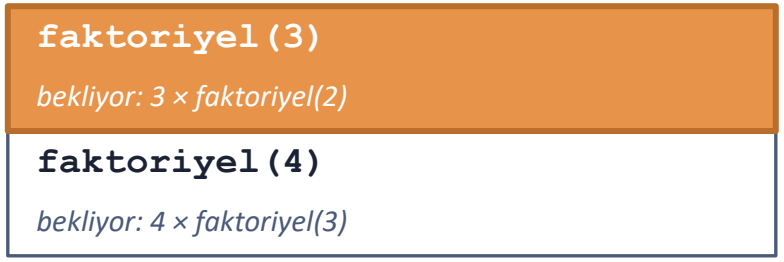
faktoriyel(4) çağrılıyor



main() faktoriyel(4)'ü çağırır. Yeni bir frame PUSH edilir. n=4 olduğu için base case değil → 4 * faktoriyel(3) beklenir.

Call Stack Büyüyor — Adım 2 / 5

faktoriyel(3) çağrılıyor



faktoriyel(4), kendi sonucunu hesaplayabilmek için faktoriyel(3)'ü çağırır. faktoriyel(4)'ün frame'i STACK'TE BEKLER, kaybolmaz.

Call Stack Büyüyor — Adım 3 / 5

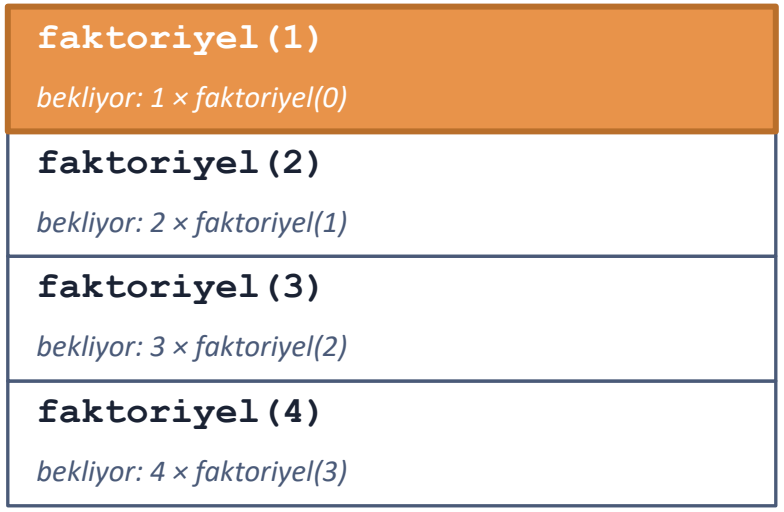
faktoriyel(2) çağrılıyor



Stack şimdi 3 frame derin. Her frame KENDİ n değerini ve bekleyen işlemini ayrı ayrı saklar.

Call Stack Büyüyor — Adım 4 / 5

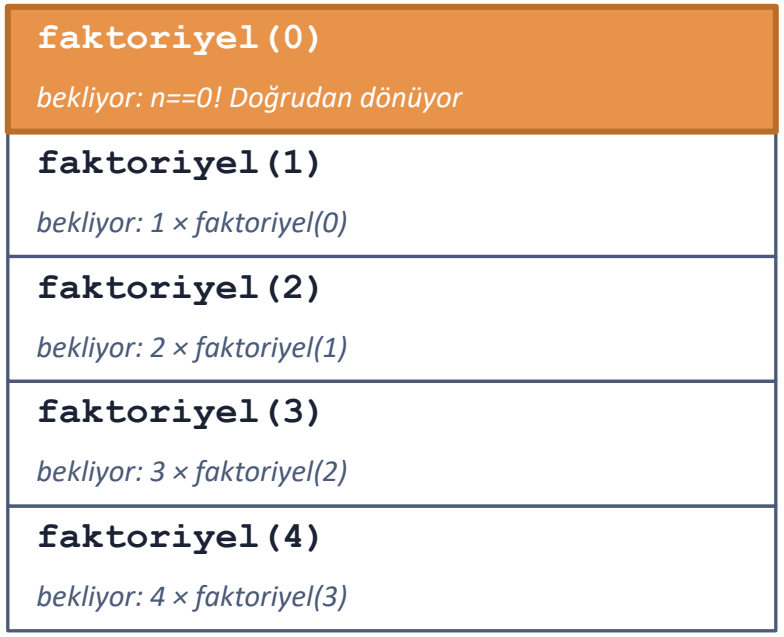
faktoriyel(1) çağrılıyor



Base case'e neredeyse ulaşıldı — bir sonraki çağrı faktoriyel(0) olacak.

Call Stack Büyüyor — Adım 5 / 5

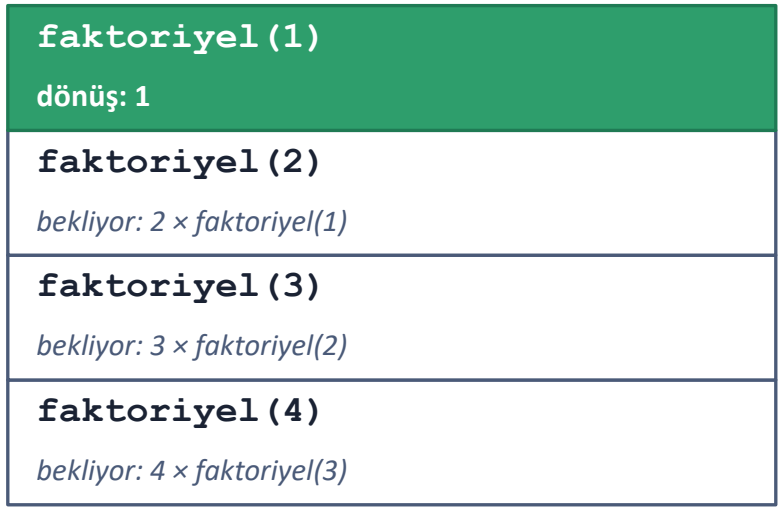
faktoriyel(0) çağrılıyor — BASE CASE'e ulaşıldı



BASE CASE! faktoriyel(0), artık YENİ bir çağrı YAPMAZ — doğrudan 1 döndürür. Stack en derin noktasına ulaştı: 5 frame.

Geri Açılma (Unwinding) — Adım 1 / 5

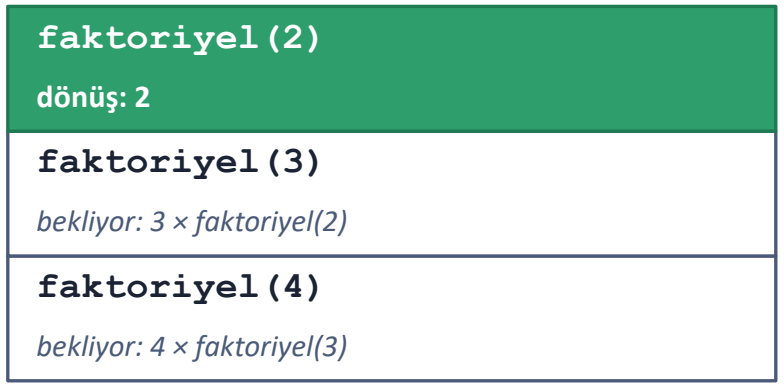
faktoriyel(0) sonucunu döndürüp POP edildi



faktoriyel(0)=1 döndü ve POP edildi (artık stack'te yok). faktoriyel(1), 1 × 1 = 1 hesaplayıp KENDİ sonucunu belirledi.

Geri Açılma (Unwinding) — Adım 2 / 5

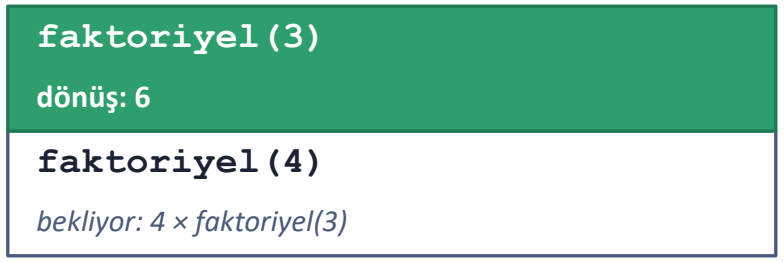
faktoriyel(1) sonucunu döndürüp POP edildi



faktoriyel(1)=1 döndü ve POP edildi. faktoriyel(2), 2 × 1 = 2 hesaplayıp KENDİ sonucunu belirledi.

Geri Açılma (Unwinding) — Adım 3 / 5

faktoriyel(2) sonucunu döndürüp POP edildi



faktoriyel(2)=2 döndü ve POP edildi. faktoriyel(3), 3 × 2 = 6 hesaplayıp KENDİ sonucunu belirledi.

Geri Açılma (Unwinding) — Adım 4 / 5

faktoriyel(3) sonucunu döndürüp POP edildi



faktoriyel(3)=6 döndü ve POP edildi. faktoriyel(4), 4 × 6 = 24 hesaplayıp KENDİ (ve son) sonucunu belirledi.

Geri Açılma (Unwinding) — Adım 5 / 5

faktoriyel(4) sonucunu döndürüp POP edildi — Stack BOŞ

(stack boş)

```
main() faktoriyel(4) = 24 sonucunu alıyor
```

Tüm frame'ler pop edildi, call stack başladığı gibi BOŞ. Sonuç (24), en dıştaki çağrıya kadar KATMAN KATMAN taşındı.

Faktöriyel — Karmaşıklık Analizi

Ölçüt	Değer	Açıklama
Zaman Karmaşıklığı	O(n)	n çağrı yapılır, her biri O(1) iş yapar
Alan Karmaşıklığı (call stack)	O(n)	Aynı anda en fazla n+1 frame stack'te bulunur

En kötü durum, en iyi durumla AYNIDIR: faktoriyel(n) her zaman TAM OLARAK n çağrı yapar (koşula bağlı dallanma yoktur). Alan karmaşıklığı özellikle önemlidir: her özyinelemeli çağrı GERÇEK bellek (stack frame) kullanır. Büyük n değerleri (ör. n=100.000) için bu, stack overflow'a yol açabilir.

Ek Not: 32-bit işaretli int türünde 12! hesaplanabilir; ancak 13! taşmaya neden olur. Daha büyük değerler için daha geniş bir veri türü veya özel sayısal yöntemler gerekir.

Fibonacci — Tanım

Örnek 2

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0, \quad F(1) = 1 \qquad \text{(iki base case)}$$

Faktöriyelden farkı: Fibonacci'de İKİ base case ve İKİ recursive çağrı vardır — bu, ÇAĞRI AĞACINI dallı hale getirir.

Fibonacci — Pseudocode

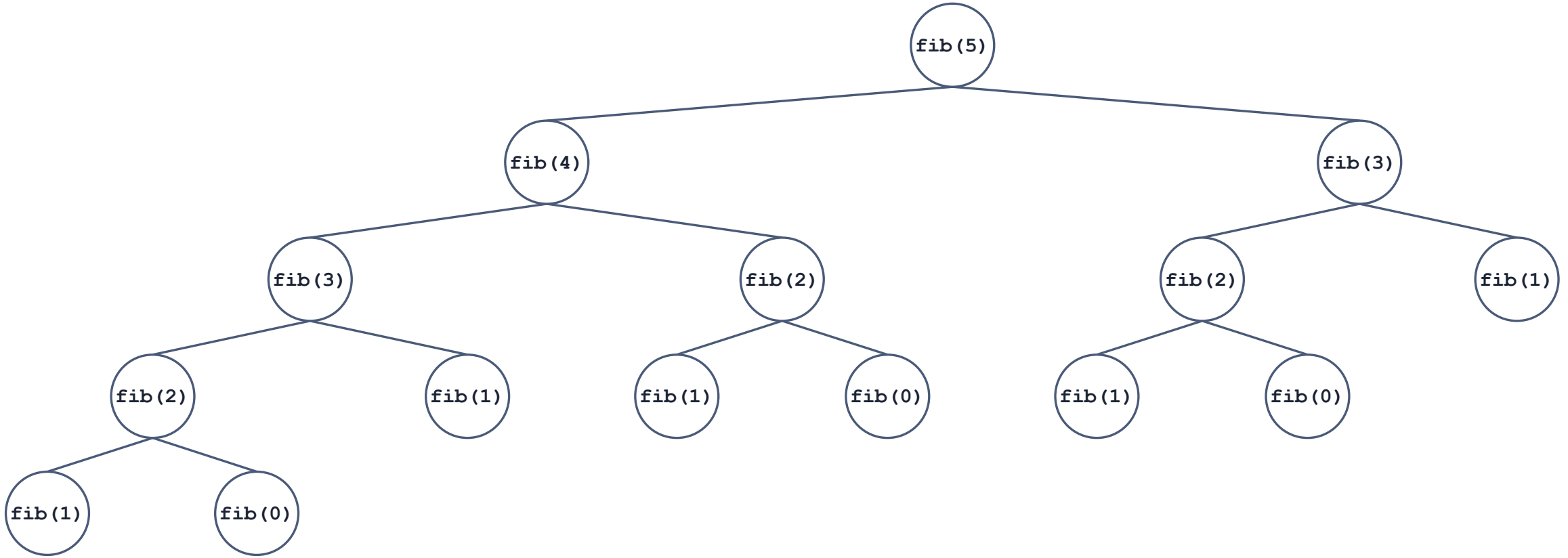
```
Fonksiyon Fib(n)
    eğer n ≤ 1
        dön n                              // base case (n=0 veya
n=1)
    dön Fib(n - 1) + Fib(n - 2)            // recursive case
```

Fibonacci — C Kodu

```c
int fib(int n) {
    if (n <= 1) {
        return n;                    // base case
    }
    return fib(n - 1) + fib(n - 2);   // recursive case
}
```

Fibonacci Çağrı Ağacı — fib(5)

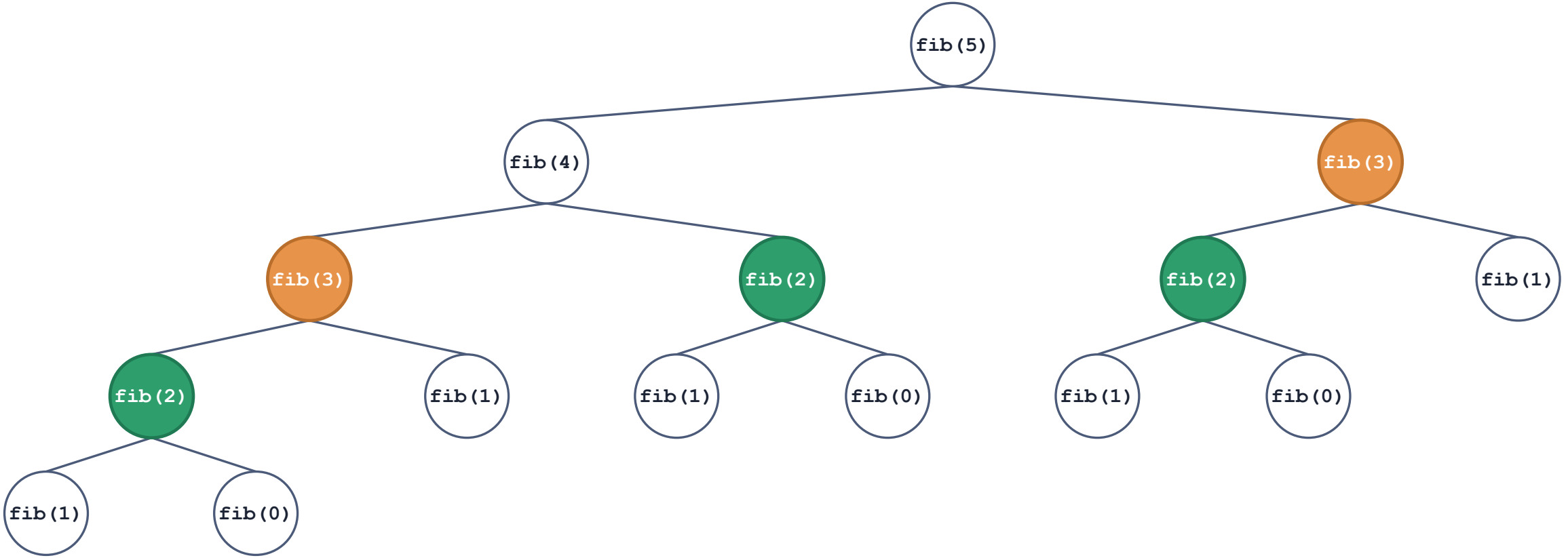
Genel Görünüm — 15 toplam çağrı



Ağacın şekli, her çağrının İKİ alt çağrı yaptığını gösterir. Toplam 15 çağrı yapılır ama sonuç sadece 5'tir.

Fibonacci Çağrı Ağacı — Tekrarlanan Hesaplamalar

fib(3) 2 kez, fib(2) 3 kez, fib(1) 5 kez YENİDEN hesaplanıyor



fib(3) — 2 kez (turuncu) **fib(2) — 3 kez (yeşil)**

Aynı değer, BAĞLAMSIZ olarak (önceki sonuç HATIRLANMADAN) defalarca yeniden hesaplanıyor — verimsizliğin kaynağı budur.

Fibonacci — Neden Bu Kadar Yavaş?

n	Çağrı Sayısı
5	15
10	177
20	21.891
30	2.692.537

Ağac derinliği n arttıkça, ağaç GENİŞLİĞİ (çağrı sayısı) YAKLAŞIK OLARAK İKİYE KATLANIR — çünkü her düğüm İKİ yeni çağrı doğurur. Bu, üstel (exponential) büyümedir. n=30 için 2.5 milyondan fazla çağrı, ama sonuç sadece TEK bir sayıdır (832.040).

Fibonacci — Karmaşıklık Analizi

$$O(2^n) \quad \text{(yaklaşık)}$$

Çağrı ağacının her seviyesinde düğüm sayısı KATLANARAK artar (her düğüm 2 çocuk doğurur), ağaç derinliği n'dir. Tam karmaşıklık $O(\phi^n)$ şeklindedir ($\phi \approx 1.618$, altın oran), ama $O(2^n)$ üst sınır olarak yaygın kullanılır. Alan karmaşıklığı ise sadece O(n) — çünkü call stack derinliği ağac GENİŞLİĞİNE değil, ağac DERİNLİĞİNE bağlıdır.

İlginç detay: zaman üsteldir ama alan sadece doğrusaldır — stack aynı anda EN FAZLA n derinliğe iner.

Fibonacci — Pratik Not

- **Naive özyinelemeli Fibonacci, sadece ÖĞRETİCİ bir örnektir — özyinelemenin hem gücünü hem de TEHLİKESİNİ (tekrarlanan hesaplama) gösterir.**
- Gerçek uygulamalarda, hesaplanan değerleri SAKLAYARAK (memoization) veya döngü ile (iterative) çözülür — bu, ileri veri yapıları derslerinin konusudur.

Bu haftaki amaç, Fibonacci'yi VERİMLİ çözmek değil — özyinelemenin çağrı ağacı üzerinden nasıl çalıştığını ve neden dikkatli kullanılması gerektiğini görmektir.

Doğrusal Toplam — Tanım

Örnek 3

```
toplam(n) = n + toplam(n - 1)
```

```
toplam(0) = 0      (base case)
```

Bu, faktöriyelle BİREBİR AYNI çağrı yapısına sahiptir — sadece çarpma yerine toplama kullanılır. 1'den n'e kadar olan sayıların toplamını hesaplar.

Doğrusal Toplam — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon Toplam(n)
    eğer n == 0
        dön 0
    dön n + Toplam(n - 1)
```

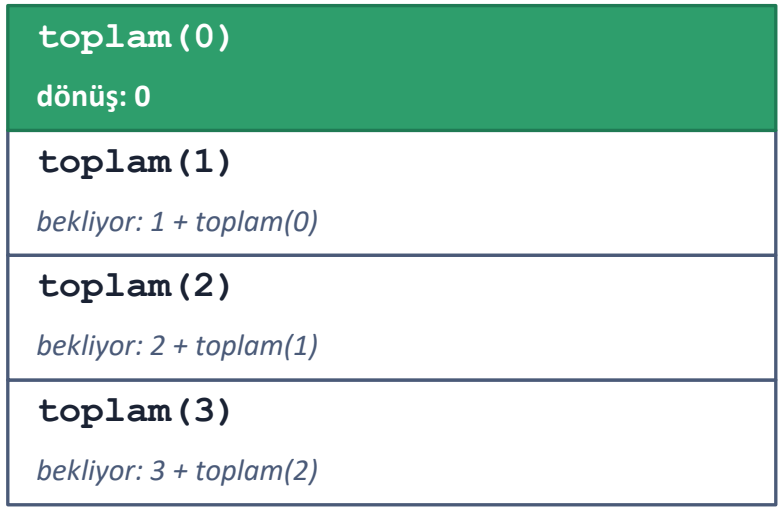
C Kodu

```c
int toplam(int n) {
    if (n == 0) {
        return 0;
    }
    return n + toplam(n - 1);
}
```

Bir dizinin ilk n elemanının toplamı için de aynı mantık kullanılır: diziToplam(arr, n) = arr[n-1] + diziToplam(arr, n-1), base case: diziToplam(arr, 0) = 0.

Toplam — Call Stack Benzerliği

toplam(3) çağrılıyor — en derin nokta



Aynı faktoriyel yapısı: 4 frame, base case'te resolved, şimdi 0+1=1, 1+2=3, 3+3=6 şeklinde geri açılacak.

Doğrusal Toplam — Karmaşıklık Analizi

Ölçüt	Değer
Zaman Karmaşıklığı	O(n)
Alan Karmaşıklığı (call stack)	O(n)

Faktöriyel ile TAMAMEN AYNI karmaşıklık profili — çünkü çağrı yapısı aynıdır (tek recursive çağrı, n ile doğrusal azalan parametre). Bu, doğrusal özyinelemenin (linear recursion) genel bir özelliğidir.

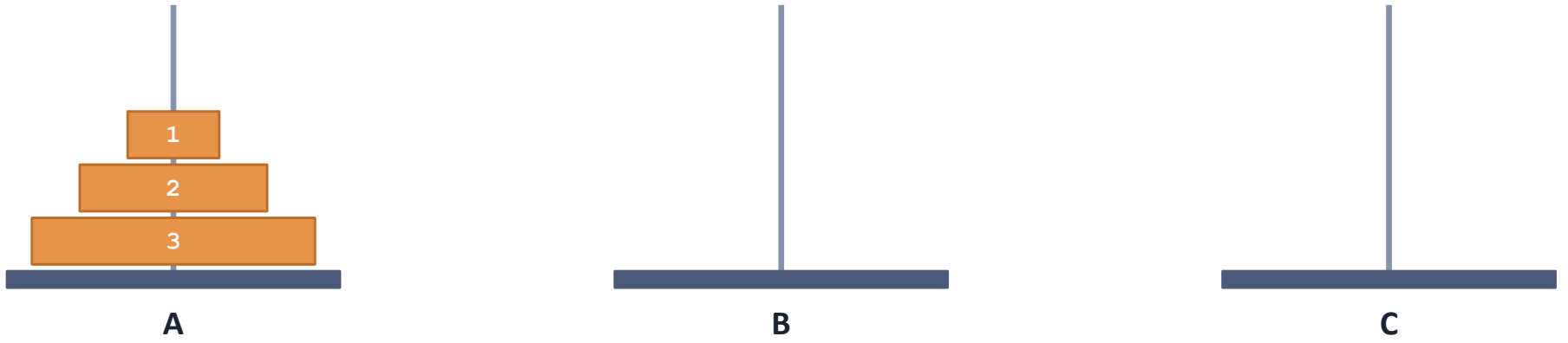
Hanoi Kuleleri — Problem Tanımı ve Kuralları

Örnek 4

- 3 çubuk (A, B, C) ve farklı boyutlarda n disk vardır. Başlangıçta tüm diskler A çubuğunda, BÜYÜKTEN KÜÇÜĞE sıralıdır.
- **AMAÇ: tüm diskleri A'dan C'ye taşımak.**
- KURAL 1: Her seferinde SADECE TEK disk taşınabilir.
- **KURAL 2: Büyük bir disk, KÜÇÜK bir diskin üstüne KONULAMAZ.**

Bu haftaki en zorlu örnek: çözüm doğası gereği özyinelemelidir ve iki AYRI recursive çağrı içerir.

Hanoi — 3 Disk Başlangıç Durumu

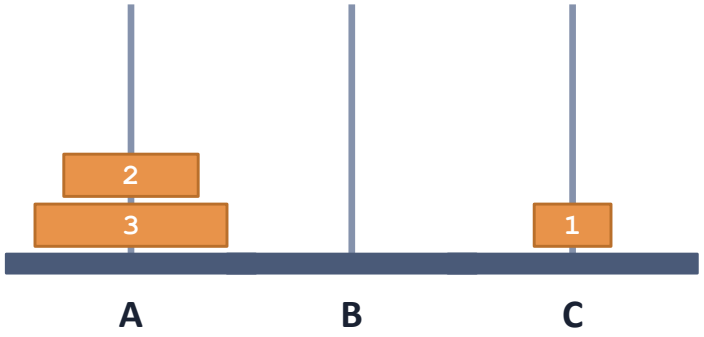


Hedef: 3 disk 7 hamlede A'dan C'ye taşınacak (2^3-1=7). Sonraki slaytlarda bu 7 hamleyi adım adım izleyeceğiz.

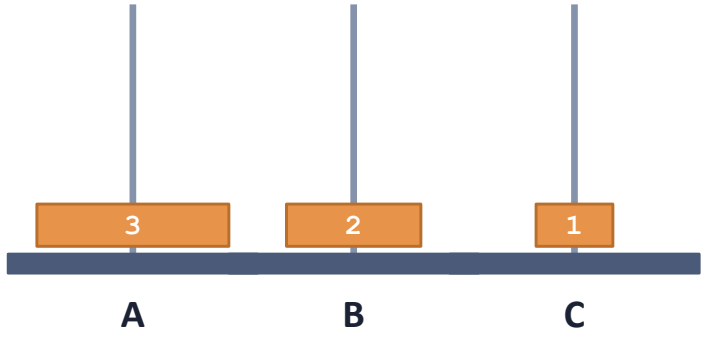
Hanoi — Hamle 1–3

n-1 diski (1,2) yardımcı çubuğa (B) taşı

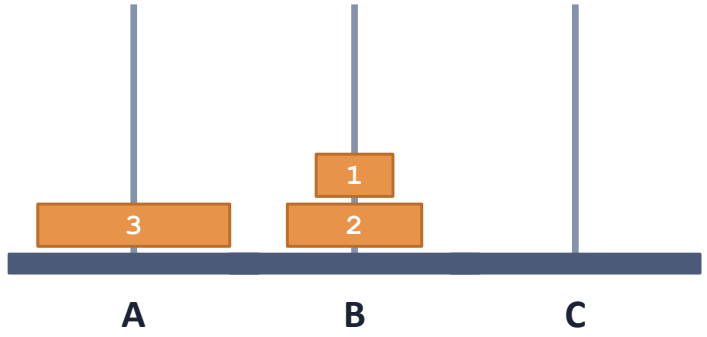
Hamle 1: disk 1, A→C



Hamle 2: disk 2, A→B



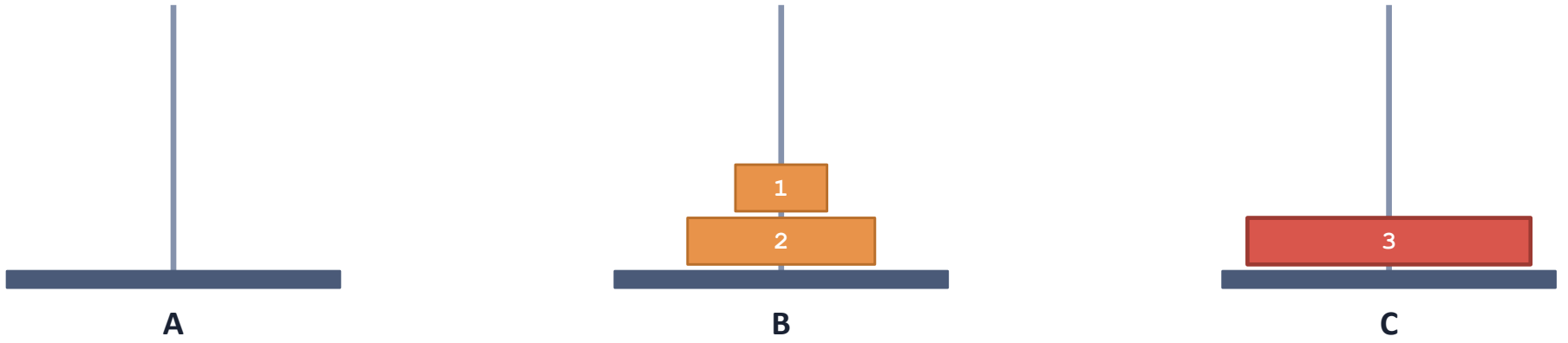
Hamle 3: disk 1, C→B



Bu 3 hamle, tam olarak Hanoi(2, A, B, C) alt problemini çözer — 2 disk A'dan B'ye taşınır.

Hanoi — Hamle 4

En büyük disk (3) doğrudan hedefe (C) taşınıyor

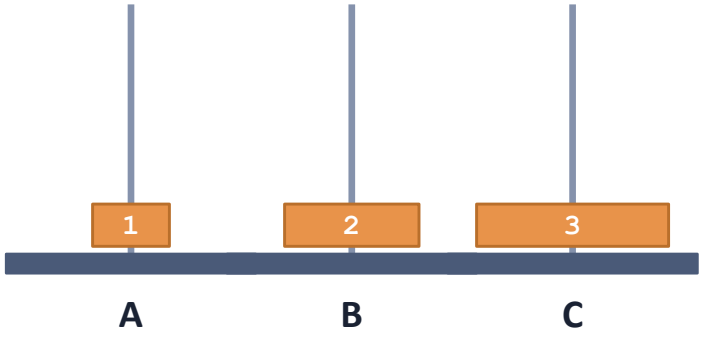


Bu, TÜM çözümün KİLİT hamlesidir: en büyük disk artık doğru yerdedir ve BİR DAHA hiç hareket ettirilmeyecektir.

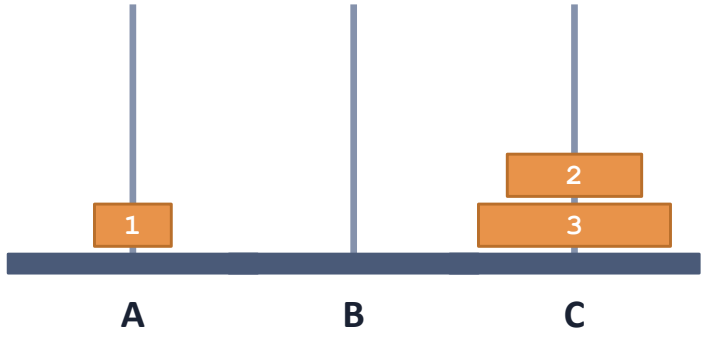
Hanoi — Hamle 5–7

n-1 diski (1,2) yardımcı çubuktan (B) hedefe (C) taşı

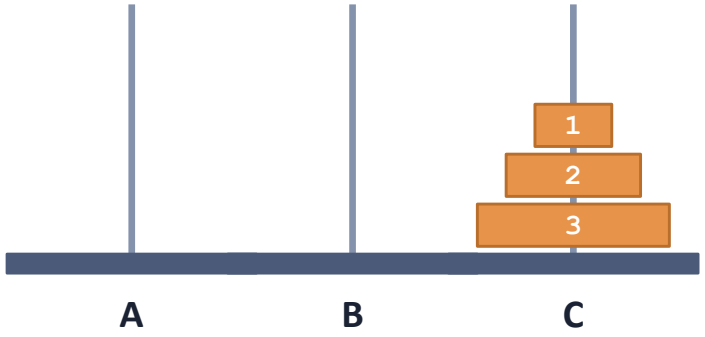
Hamle 5: disk 1, B→A



Hamle 6: disk 2, B→C



Hamle 7: disk 1, A→C



Hamle 7'de çözüm TAMAMLANDI: tüm 3 disk C çubuğunda, doğru sırada. Toplam 7 hamle kullanıldı.

Hanoi — Recursive Mantık

Hanoi(n, kaynak, hedef, yardımcı)

1

`Hanoi(n-1, kaynak, YARDIMCI, hedef)`

n-1 diski kaynaktan yardımcıya taşı (hedef, geçici olarak yardımcı rolünde)

2

`1 diski kaynak→hedef taşı`

En büyük disk doğrudan hedefe gider (KILIT hamle, slayt 44)

3

`Hanoi(n-1, yardımcı, hedef, KAYNAK)`

n-1 diski yardımcıdan hedefe taşı (kaynak, geçici olarak yardımcı rolünde)

Hanoi — Pseudocode

```
Fonksiyon Hanoi(n, kaynak, hedef, yardimci)
    eğer n == 0
        dön                                    // base case: taşınacak disk yok
    Hanoi(n - 1, kaynak, yardimci, hedef)        // 1) n-1'i yardımcıya
    “disk n'i kaynaktan hedefe taşı” yazdır     // 2) en büyüğü hedefe
    Hanoi(n - 1, yardimci, hedef, kaynak)        // 3) n-1'i hedefe
```

Hanoi — C Kodu

```c
void hanoi(int n, char kaynak, char hedef, char yardimci) {
    if (n == 0) {
        return;                 // base case: taşınacak disk yok
    }
    hanoi(n - 1, kaynak, yardimci, hedef);       // 1) n-1'i yardımcıya
    printf("disk %d: %c -> %c\n", n, kaynak, hedef);   // 2) en büyüğü hedefe
    hanoi(n - 1, yardimci, hedef, kaynak);       // 3) n-1'i hedefe
}
```

Hanoi — Hamle Sayısı Analizi

- T(n) = n disk için gereken TOPLAM hamle sayısı olsun.

$$T(n) = 2 \times T(n-1) + 1, \qquad T(0) = 0$$

Neden? Recursive mantığın 1. ve 3. adımları HER BİRİ T(n-1) hamle gerektirir (n-1 diski taşımak), ve 2. adım TEK bir hamledir (en büyük diski taşımak). Toplam: T(n-1) + 1 + T(n-1) = 2T(n-1) + 1.

Hanoi — T(n) Çözümü: $2^n - 1$

n	T(n) = 2T(n-1)+1	$2^n - 1$
0	0	0
1	1	1
2	3	3
3	7	7
4	15	15

Tümevarımla (induction) kanıtlanabilir: $T(n)=2^n-1$ varsayılırsa, $T(n+1)=2T(n)+1=2(2^n-1)+1=2^{n+1}-1$ — formül kendini doğrular. n=3 için $2^3-1=7$, tam olarak biraz önce izlediğimiz 7 hamleye eşit.

Hanoi — Karmaşıklık Analizi

Ölçüt	Değer	Açıklama
Zaman Karmaşıklığı	$O(2^n)$	Toplam hamle sayısı 2^n-1, üstel büyüme
Call Stack Derinliği	O(n)	Aynı anda en fazla n frame stack'te (Fibonacci ile aynı durum)

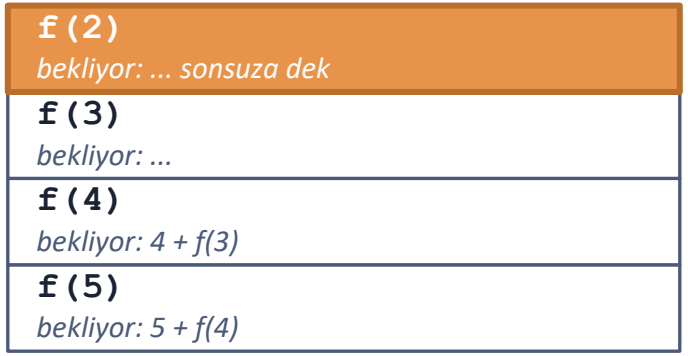
Önemli ayrım: TOPLAM çağrı sayısı üsteldir ($O(2^n)$) ama call stack'in AYNI ANDA tuttuğu frame sayısı sadece n'dir (doğrusal) — çünkü 1. recursive çağrı TAMAMEN biter (tüm alt-çağrılarıyla) ÖNCE 2. recursive çağrı başlar. Bu, Fibonacci'nin çağrı ağacıyla AYNI genel prensiptir (35. slayt).

64 disklik efsanevi "Hanoi Kulesi" problemi $2^{64}-1$ hamle gerektirir — saniyede 1 hamleyle bile evrenin yaşından uzun sürer!

Yaygın Hata 1: Base Case Yok

```
int f(int n) {
    return n + f(n - 1);     // BASE CASE YOK!
}
```

f(n), HİÇBİR n değeri için durmaz — n azalıp eksiye geçse bile çağrı devam eder.



Yaygın Hata 1 — Neden Stack Overflow?

- **Her çağrı, gerçek belleği (stack frame) kullanır — bellek SONSUZ DEĞİLDİR.**
- Base case olmadığı için program KENDİLİĞİNDEN durmaz.
- Bellek tükendiğinde, işletim sistemi programı “Stack Overflow” hatasıyla SONLANDIRIR.

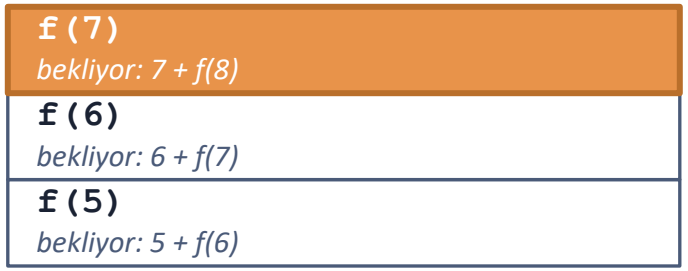
```
Düzeltme: HER ZAMAN, girdiyi KESİNLİKLE base case'e yaklaştıran bir koşul ekleyin:

int f(int n) {
    if (n <= 0) return 0;   // base case eklendi
    return n + f(n - 1);
}
```

Yaygın Hata 2: Base Case'e Ulaşılamıyor

```
int f(int n) {
    if (n == 0) return 0;      // base case VAR
    return n + f(n + 1);       // ama n ARTIYOR, azalmıyor!
}
```

Base case (n==0) VAR ama f(5) çağrıldığında n; 5,6,7,8,... şeklinde ARTAR — 0'dan UZAKLAŞIR, asla ulaşılmaz.



Yaygın Hata 2 — Analiz

- **Base case'in VAR OLMASI YETMEZ — girdinin base case'e DOĞRU YÖNDE ilerlediğinden EMİN olunmalıdır.**
- Bu örnekte düzeltme basittir: f(n+1) yerine f(n-1) yazılmalıydı.
- *İlginç ve sinsi bir detay: bu HATALI kod (f(n+1) ile), n=-3 gibi NEGATİF bir değerle çağrılırsa -3→-2→-1→0 şeklinde İLERLEYİP base case'e ULAŞIR ve çalışır görünür — hata sadece POZİTİF başlangıçlarda ortaya çıkar, bu da onu fark etmeyi zorlaştırır.*

Ders: Sadece “base case var mı?” diye sormak yetmez — “HER çağrıda girdi bu base case'e doğru mu ilerliyor?” diye de sorulmalıdır.

Hata Önleme Kontrol Listesi

- ☑ En az bir base case var mı?
- ☑ Her recursive çağrı, girdiyi base case'e DOĞRU YÖNDE değiştiriyor mu?
- ☑ Base case, TÜM geçerli girdiler için ULAŞILABİLİR mi?
- ☑ Recursive çağrı, ORİJİNAL problemden gerçekten DAHA KÜÇÜK bir alt problemi mi çözüyor?

Recursion vs Iteration — Karşılaştırma

Ölçüt	Özyineleme (Recursion)	Yineleme (Iteration)
Kod Okunabilirliği	Genelde DAHA SADE (özellikle ağaç/bölünmüş problemlerde)	Genelde daha uzun, ama daha DOĞRUSAL
Bellek Kullanımı	Her çağrı için stack frame gerekir	Probleme ve kullanılan ek veri yapılarına bağlıdır; basit döngülerde O(1) olabilir.
Call Stack Maliyeti	VAR — derinlik kadar frame birikir	YOK
Hız	Fonksiyon çağrı ek yükü nedeniyle genelde biraz DAHA YAVAŞ	Genelde biraz DAHA HIZLI
Stack Overflow Riski	VAR (çok derin çağrılarda)	Yinelemenin kendisi çağrı yığınını derinleştirmez; ancak uygulamaya bağlı başka bellek sınırları bulunabilir.
En Uygun Olduğu Yapı	Ağaç/graf gezintisi, böl-ve-yönet problemleri	Basit doğrusal tekrarlar, sayaç tabanlı döngüler

Hangi Durumda Hangisi Daha Doğal?

Özyineleme Daha Doğaldır

- Hanoi Kuleleri (doğası gereği özyinelemeli tanımlanır)
- Ağaç yapılarında gezinme (gelecek haftalarda göreceğiz)
- Böl-ve-yönet algoritmaları (ör. hızlı sıralama)

Yineleme Daha Doğaldır

- Bir dizinin toplamını/ortalamasını hesaplama
- Sabit sayıda tekrar gerektiren basit döngüler
- Performansın KRİTİK olduğu, derinliği önceden bilinmeyen durumlar

“Özyineleme her zaman daha iyidir” ya da “yineleme her zaman daha iyidir” gibi bir genelleme YOKTUR — seçim, problemin doğasına ve derinlik/performans gereksinimlerine bağlıdır.

Tail Recursion — Kısa Tanıtım

İleri Düzey Ek Materyal

```
long faktoriyel_tail(int n, long biriken) {
    if (n == 0) {
        return biriken;
    }
    return faktoriyel_tail(n - 1, n * biriken);   // çarpma ÖNCE yapılıyor
}
// Çağrı: faktoriyel_tail(4, 1)
```

Recursive çağrı, fonksiyonun EN SON işlemidir — çağrı döndükten sonra YAPILACAK BAŞKA İŞLEM YOKTUR (çarpma önceden, parametre olarak yapılır).

ÖNEMLİ: C derleyicileri TAIL-CALL OPTIMIZATION yapmak ZORUNDA DEĞİLDİR. Bu yüzden tail recursion kullanmak, stack kullanımının KESİNLİKLE ortadan kalkacağı anlamına GELMEZ — derleyiciye ve optimizasyon ayarlarına bağlıdır.

Haftalık Özet

- Özyinelemeyi (bir fonksiyonun kendini çağırmasını) ve base case/recursive case ikilisini öğrendik.
- Faktöriyel örneğinde call stack'in NASIL büyüdüğünü ve NASIL geri açıldığını adım adım gördük.
- Fibonacci çağrı ağacıyla, naive özyinelemenin tekrarlanan hesaplama yüzünden üstel olarak yavaşladığını gördük.
- Hanoi Kuleleri'nin TAM çözümünü, $T(n)=2^n-1$ kanıtıyla birlikte işledik.
- Base case eksikliği ve yanlış yönde ilerleme gibi yaygın hataları tanıdık.
- Recursion ve iteration'ı doğru kriterlerle, genelleme yapmadan karşılaştırdık.

Gelecek Hafta Önizleme

6. Hafta: Kuyruklar (Queues)

- Kuyruk (queue) ADT: enqueue, dequeue, front, isEmpty, isFull
- **FIFO (İlk Giren İlk Çıkar) prensibi — 4. haftada LIFO ile karşılaştırdığımız kavram.**
- Dizi ile ve bağlı listeyle gerçekleştirim
- Dairesel kuyruk (circular queue) ile bellek verimliliği
- Gerçek uygulamalar: yazdırma kuyrukları, işlemci zamanlaması, BFS (Breadth-First Search)

Hazırlık: Bu haftaki stack ADT'sini (push/pop, LIFO) aklınızda tutun — kuyruk, aynı temel ADT fikrinin FARKLI bir erişim kuralıyla (FIFO) versiyonudur.

Kavrama Soruları / Mini Quiz

1) faktoriyel(3) çağrıldığında call stack'te AYNI ANDA en fazla kaç frame bulunur?

2) fib(4) çağrıldığında, fib(1) toplam kaç kez yeniden hesaplanır?

3) 'int f(int n) { if (n==10) return 0; return n + f(n-1); }' fonksiyonu f(3) ile çağrılırsa ne olur? Neden?

4) Hanoi(4) için kaç hamle gerekir? Formülü kullanarak gösteriniz.

Haftalık Ödev

- 1) Recursive faktoriyel fonksiyonunu C'de yazınız ve faktoriyel(0) ile faktoriyel(10) arası tüm değerleri test ediniz.
- 2) Bir dizinin ilk n elemanının toplamını hesaplayan recursive bir fonksiyon yazınız.
- 3) fib(6) için TAM çağrı ağacını elle çiziniz — tekrarlanan değerleri işaretleyiniz.
- 4) Hanoi(4) için gereken TÜM hamleleri (15 hamle) sırayla listeleyiniz.
- **5) Aşağıdaki fonksiyonu yineleyerek (iterative) yeniden yazınız ve iki versiyonu karşılaştırınız: int recTopla(int n) { if (n==0) return 0; return n + recTopla(n-1); }**

Teslim: Bir sonraki ders öncesi, C kaynak dosyaları + elle çiziminiz (fotoğraf/tarama) içeren bir PDF olarak.

Kaynakça

Kaynak	İlgili Bölüm(ler)	Sayfa Aralığı
Horowitz, Sahni, Anderson-Freed — Fundamentals of Data Structures in C	Recursion / Chapter 3-4 ilgili bölümler	İlgili sayfalar
Thareja — Data Structures Using C	Chapter: Recursion	İlgili sayfalar

Not:

Call stack görselleri, Fibonacci çağrı ağacı diyagramı, Hanoi adım adım çözümü ve hata örnekleri özgün olarak hazırlanmıştır; kaynaklardaki ilgili bölümler kavramsal çerçeve için referans alınmıştır.